

Significance

No matter which programming language you use, the end result is machine code, or assembly language. Knowing something about the end result of your code is of obvious benefit.

Think of the difference between knowing how to drive a car, and knowing how an engine, and other components of the car work. It's possible to know how to drive a car without knowing a thing about spark plugs and brake oil, but understanding how a car works puts you in a better position, especially when the car breaks down. In our case, while debugging software you can now peek at the instructions behind your instructions when you learn assembly.

Given how low-level assembly language is, it is also possible to improve performance when coding directly in assembly. However with the complexity of software today it's infeasible to develop in assembly, especially since such software would need to be rewritten for every platform it is intended to run on.

What is commonly done is that, performance-sensitive portions of higher-level code is written in assembly. These are usually sections of the code that are called thousands or even millions of times.

For example, a game might need to constantly evaluate the distance between two points. This piece of code might be called thousands of times in rendering a single frame. It makes sense to write this in assembly and gain a huge boost in performance as a result. This code will need to be rewritten for each platform though. Speaking of platforms and their differences, there are two major schools of CPU design that have a large impact on their assembly language as well. These are Reduced Instruction Set Computing (RISC) and Complex Instruction Set Computing (CISC).

The RISC design strategy is to have fewer possible instructions, but have them execute faster. With CISC the aim is to have fewer instructions that are more powerful. For example, a CISC processor might have an instruction



You can still buy an 8085 microprocessor training kit that lets you program the CPU by sending it single instructions.